ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Сбор рыночных данных: Binance & Bybit → Redis

Простые скрипты — максимум надёжности

Версия: 2.1 (no Docker)
Дата: Март 2026

Содержание

Для обновления: ПКМ на оглавление → Обновить поле.

1. Цель проекта

Минималистичная, но отказоустойчивая система сбора рыночных данных (best bid/ask) с бирж Binance и Bybit через WebSocket. Данные записываются в Redis. Два вспомогательных скрипта обеспечивают мониторинг: обнаружение stale-символов и отслеживание задержек.

Принципы:

- Максимальная простота: один файл = один скрипт, минимум абстракций.
- Отказоустойчивость: бесконечный reconnect с backoff, буфер при Redis down.
- Хорошие логи: structlog JSON с контекстом (exchange, market, symbol, conn_id).
- Нет лишних зависимостей: Python 3.11+, websockets, redis[hiredis], orjson, structlog.
- Списки символов готовые и нормализованные — лежат в проекте.

2. Состав проекта

2.1. Структура файлов

```
market-data/
├── binance_spot.py
├── binance_futures.py
├── bybit_spot.py
├── bybit_futures.py
├── stale_monitor.py
├── latency_monitor.py
├── common.py                  # общие функции (Redis, логи, загрузка
символов)
├── requirements.txt
├── .env.example
└── dictionaries/
    └── subscribe/
        ├── binance/
        │   ├── binance_spot.txt
        │   └── binance_futures.txt
        └── bybit/
            ├── bybit_spot.txt
            └── bybit_futures.txt
```

7 файлов кода + 1 общий модуль. Никаких пакетов, абстрактных классов, фабрик. Каждый коллектор — самодостаточный скрипт с одной точкой входа.

2.2. common.py — общий модуль

Единственный разделяемый модуль. Содержит:

- setup_logging(service_name) → настройка structlog (JSON, уровень из env).
- load_symbols(filepath) → list[str] — чтение файла, strip, upper, дедупликация, skip пустых строк и комментариев (#). При пустом результате — sys.exit(1).
- get_redis() → redis.asyncio.Redis — создание клиента с hiredis, retry_on_error=True.
- now_ms() → int — текущий timestamp в миллисекундах.
- Константы: REDIS_URL, REDIS_KEY_TTL (120 сек), общие env-переменные.

2.3. requirements.txt

```
websockets>=12.0
redis[hiredis]>=5.0
orjson>=3.9
structlog>=24.0
python-dotenv>=1.0
```

2.4. Запуск

Каждый скрипт запускается напрямую как отдельный процесс:

```
python binance_spot.py
python binance_futures.py
python bybit_spot.py
python bybit_futures.py
python stale_monitor.py
python latency_monitor.py
```

Рекомендуемый способ управления процессами — systemd, supervisord, или tmux/screen. Каждый скрипт работает бесконечно до получения SIGTERM/SIGINT.

3. WS-коллекторы: общая логика

3.1. Жизненный цикл (одинаков для всех 4 скриптов)

1. Загрузка символов из файла через common.load_symbols().
2. Разбиение на chunks по 150 символов (запас от лимита 200).
3. Для каждого chunk — отдельная asyncio.Task (ws_worker).
4. ws_worker: цикл connect → subscribe → read messages → write redis → reconnect on error.
5. Главный цикл: asyncio.gather(*workers), ловит SIGTERM/SIGINT для graceful shutdown.

3.2. Reconnect-стратегия

Бесконечный цикл с exponential backoff + jitter:

```
delay = min(2 ** attempt, 60) + random.uniform(0, 1)
```

- Начальная задержка: 1 сек.
- Максимальная задержка: 60 сек.
- После успешного подключения и получения первого сообщения — сброс счётчика.
- Каждый reconnect логируется: WARNING с attempt, delay, error.
- Скрипт никогда не завершается сам — только по SIGTERM.

3.3. Запись в Redis

Одно обновление = один Redis Pipeline из 3 команд:

```
pipe = redis.pipeline()
pipe.hset(key, mapping={...})       # данные
pipe.expire(key, TTL)               # авто-удаление при stale
pipe.publish(channel, payload)      # уведомление consumer-ов
await pipe.execute()
```

Формат ключа:

```
md:{exchange}:{market}:{symbol}
```

Примеры: md:binance:spot:BTCUSDT, md:bybit:futures:ETHUSDT

Pub/Sub канал:

```
md:updates:{exchange}:{market}
```

3.4. Поля Redis Hash

Поле	Тип	Описание
bid	str	Best bid price
bid_qty	str	Best bid quantity
ask	str	Best ask price
ask_qty	str	Best ask quantity
last	str	Last trade price (если есть)
ts_exchange	str (int ms)	Timestamp от биржи (0 если нет)
ts_received	str (int ms)	Timestamp получения WS-сообщения
ts_redis	str (int ms)	Timestamp записи в Redis

3.5. Обработка ошибок

Ситуация	Действие	Лог
WS disconnect (любой)	Reconnect с backoff	WARNING
JSON parse error	Skip сообщение, продолжить	WARNING
Redis connection lost	Retry каждые 2 сек, drop сообщений с логом	ERROR
Неизвестный формат msg	Skip, продолжить	WARNING
Файл символов не найден	Exit(1)	CRITICAL
Пустой файл символов	Exit(1)	CRITICAL

3.6. Ping / Heartbeat

Биржа	Механизм	Действие коллектора
Binance	Сервер ожидает pong. Disconnect через 10 мин без pong.	websockets автоматически отвечает pong. Доп. ping каждые 30 сек.
Bybit	Сервер шлёт ping. Ожидает pong.	Ответ: {"op":"pong"} на каждый ping. Доп. ping каждые 20 сек.

3.7. Graceful Shutdown

1. SIGTERM / SIGINT → устанавливается asyncio.Event (shutdown_event).
2. Все ws_worker-ы проверяют shutdown_event и завершают цикл чтения.
3. Закрытие WS-соединений (close code 1000).
4. Закрытие Redis-клиента.
5. Лог: "graceful_shutdown" INFO.
6. Exit(0).

4. Спецификация каждого коллектора

4.1. binance_spot.py

Параметр	Значение
WS URL	wss://stream.binance.com:9443/ws
Файл символов	dictionaries/subscribe/binance/binance_spot.txt
Подписка	{"method":"SUBSCRIBE","params":["btcusdt@bookTicker",...],"id":1}
Символы в params	lowercase + @bookTicker
Макс. подписок / conn	150 (лимит 200, запас 50)

Формат входящего сообщения:

```
{  "u":400900217, "s":"BTCUSDT", "b":"67234.50", "B":"1.234",
"a":"67234.80", "A":"0.567" }
```

Маппинг:

Поле биржи	Redis поле	Примечание
s	→ ключ	Символ (uppercase)
b	bid	
B	bid_qty	
a	ask	
A	ask_qty	
u	—	Update ID, не сохраняем (упрощение)
—	ts_exchange	= ts_received (bookTicker не содержит ts биржи)

4.2. binance_futures.py

Параметр	Значение
WS URL	wss://fstream.binance.com/ws
Файл символов	dictionaries/subscribe/binance/binance_futures.txt
Подписка	Аналогично Spot: {"method":"SUBSCRIBE","params":["btcusdt@bookTicker",...],"id":1}
Макс. подписок / conn	150

Отличие от Spot: Futures bookTicker содержит поле E (event time ms).

```
{  "e":"bookTicker", "u":400900217, "E":1710412800000, "T":1710412799999,
```

```
"s":"BTCUSDT", "b":"67234.50", "B":"1.234", "a":"67234.80", "A":"0.567" }
```

Маппинг: аналогичен Spot, но ts_exchange = E (event time от биржи).

4.3. bybit_spot.py

Параметр	Значение
WS URL	wss://stream.bybit.com/v5/public/spot
Файл символов	dictionaries/subscribe/bybit/bybit_spot.txt
Подписка	{"op":"subscribe","args":["tickers.BTCUSDT","tickers.ETHUSDT",...]}
Символы в args	"tickers." + SYMBOL (uppercase)
Макс. подписок / conn	150

Формат входящего сообщения:

```
{ "topic":"tickers.BTCUSDT", "type":"snapshot", "ts":1710412800000,
  "data": { "symbol":"BTCUSDT", "bid1Price":"67234.50", "bid1Size":"1.234",
            "ask1Price":"67234.80", "ask1Size":"0.567",
"lastPrice":"67234.60" } }
```

Маппинг:

Поле биржи	Redis поле
data.bid1Price	bid
data.bid1Size	bid_qty
data.ask1Price	ask
data.ask1Size	ask_qty
data.lastPrice	last
ts	ts_exchange

Важно: Bybit шлёт ping в виде JSON:

```
{"op":"ping"}  →  отвечать: {"op":"pong"}
```

А также heartbeat: {"ret_msg":"pong","op":"pong"} — это ответ сервера, игнорировать.

4.4. bybit_futures.py

Параметр	Значение

WS URL	wss://stream.bybit.com/v5/public/linear
Файл символов	dictionaries/subscribe/bybit/bybit_futures.txt
Подписка	Аналогично Bybit Spot
Макс. подписок / conn	150

Формат данных и маппинг полностью идентичен Bybit Spot (раздел 4.3).

5. stale_monitor.py

5.1. Назначение

Каждые 30 секунд сканирует Redis и находит символы, данные по которым не обновлялись более 60 секунд. Логирует найденные stale-символы и публикует алерт в Redis Pub/Sub.

5.2. Алгоритм

1. SCAN по паттерну md:*:*:* (использовать SCAN, никогда KEYS).
2. Для каждого ключа: HGET ts_redis.
3. delta = now_ms() - int(ts_redis).
4. Если delta > 60000 мс — символ stale.
5. Группировка stale по exchange:market.
6. Лог WARNING для каждого stale-символа.
7. Сводный лог: "stale_scan_complete" с total_keys, stale_count.
8. PUBLISH алерт в канал alerts:stale (JSON со списком stale).
9. Пауза 30 сек. Повторить.

5.3. Нюансы

- Grace period: первые 2 цикла (60 сек) после старта не генерировать алерты — коллекторы могут ещё не заполнить Redis.
- Recovery: если символ был stale и вернулся — лог INFO "stale_recovered".
- SCAN COUNT=500 для баланса скорости и нагрузки на Redis.
- Ключи без ts_redis (битые данные) — лог WARNING, skip.

5.4. Формат алерта (Pub/Sub: alerts:stale)

```
{
  "ts": 1710412800000,
  "stale_count": 3,
  "symbols": [
    {"key": "md:binance:spot:XYZUSDT", "age_sec": 95},
    {"key": "md:bybit:futures:ABCUSDT", "age_sec": 72}
  ]
}
```

5.5. Конфигурация (env)

Переменная	По умолчанию	Описание
REDIS_URL	redis://localhost:6379/0	
STALE_THRESHOLD_SEC	60	Порог устаревания

SCAN_INTERVAL_SEC	30	Интервал сканирования
SCAN_COUNT	500	Batch size для SCAN
LOG_LEVEL	INFO	

6. latency_monitor.py

6.1. Назначение

Непрерывно отслеживает задержки на каждом этапе: биржа → коллектор → Redis. Логирует агрегированную статистику и аномалии.

6.2. Измеряемые задержки

Метрика	Формула	Что показывает
exchange → collector	ts_received - ts_exchange	Сетевая задержка (с учётом clock skew)
collector → redis	ts_redis - ts_received	Время обработки и записи
end-to-end	ts_redis - ts_exchange	Полная задержка

6.3. Режим работы: периодический sampling

Каждые 10 секунд:

1. SCAN по паттерну md:*:*:* (до 200 ключей за цикл).
2. HMGET ts_exchange, ts_received, ts_redis.
3. Вычисление трёх дельт для каждого ключа.
4. Агрегация: min, max, avg, P95 (простой sorted percentile).
5. Лог INFO: "latency_report" с агрегатами по каждой паре exchange:market.
6. Если end-to-end > 1000ms — лог WARNING "latency_anomaly".
7. Если end-to-end > 5000ms — лог CRITICAL.
8. Если ts_received - ts_exchange < 0 — лог INFO "clock_skew_detected" (не ошибка).

6.4. Формат лога latency_report

```
{
  "event": "latency_report",
  "exchange": "binance", "market": "spot",
  "samples": 150,
  "e2e_ms": {"min": 1.2, "max": 45.3, "avg": 4.8, "p95": 12.1},
  "collector_to_redis_ms": {"min": 0.3, "max": 8.1, "avg": 1.2, "p95": 3.4},
  "anomalies": 0
}
```

6.5. Конфигурация (env)

Переменная	По умолчанию	Описание
REDIS_URL	redis://localhost:6379/0	

SAMPLING_INTERVAL_SEC	10	Интервал сканирования
SAMPLING_MAX_KEYS	200	Макс. ключей за цикл
ANOMALY_WARN_MS	1000	Порог WARNING
ANOMALY_CRIT_MS	5000	Порог CRITICAL
LOG_LEVEL	INFO	

7. Логирование

7.1. Формат: structlog JSON

```
{
  "timestamp": "2026-03-14T12:00:00.123456Z",
  "level": "info",
  "event": "ws_connected",
  "service": "binance_spot",
  "conn_id": 0,
  "symbols_count": 150
}
```

7.2. Обязательные события

Событие (event)	Уровень	Когда
collector_started	INFO	Запуск скрипта, загрузка N символов, M connections
ws_connected	INFO	WS-соединение установлено
ws_subscribed	INFO	Подписка подтверждена
ws_disconnected	WARNING	WS-соединение разорвано (reason, code)
ws_reconnecting	WARNING	Попытка reconnect (attempt, delay_sec)
ws_error	ERROR	Ошибка в WS-цикле (error message)
redis_error	ERROR	Ошибка записи в Redis
redis_recovered	INFO	Redis снова доступен после ошибки
message_parse_error	WARNING	Не удалось распарсить WS-сообщение
stale_detected	WARNING	Символ stale (key, age_sec)
stale_recovered	INFO	Символ вернулся в норму
stale_scan_complete	INFO	Цикл сканирования завершён (total, stale_count)
latency_report	INFO	Агрегаты задержек за цикл
latency_anomaly	WARNING/CRIT	Превышен порог задержки (key, e2e_ms)
clock_skew_detected	INFO	Отрицательный exchange_to_collector
graceful_shutdown	INFO	Получен сигнал, завершение

7.3. Правила логирования

- Вывод в stdout (совместимо с любым log collector).
- Уровень из env LOG_LEVEL (по умолчанию INFO).
- Никогда не логировать данные (bid/ask/price) — только метаинформацию.
- В каждом логе: service, event, timestamp. В WS-логах: conn_id.
- При ошибках: включать traceback через structlog exc_info.

8. Полная конфигурация (.env)

Переменная	По умолчанию	Описание
REDIS_URL	redis://localhost:6379/0	URL Redis
REDIS_KEY_TTL	120	TTL ключей в секундах
WS_PING_INTERVAL	30	Интервал ping (Binance=30, Bybit=20, задаётся per-script)
WS_RECV_TIMEOUT	60	Таймаут получения данных → reconnect
RECONNECT_MAX_DELAY	60	Макс. задержка backoff (секунды)
SYMBOLS_PER_CONN	150	Символов на одно WS-соединение
LOG_LEVEL	INFO	DEBUG / INFO / WARNING / ERROR / CRITICAL
STALE_THRESHOLD_SEC	60	Порог stale (stale_monitor)
SCAN_INTERVAL_SEC	30	Интервал scan (stale_monitor)
SAMPLING_INTERVAL_SEC	10	Интервал sampling (latency_monitor)
ANOMALY_WARN_MS	1000	Порог WARNING (latency_monitor)
ANOMALY_CRIT_MS	5000	Порог CRITICAL (latency_monitor)

9. Критерии приёмки

1. Запуск всех 6 скриптов — все работают без ошибок, логи в JSON.
2. redis-cli HGETALL md:binance:spot:BTCUSDT — возвращает актуальные данные.
3. Остановить binance_spot.py → stale_monitor через ~60 сек логирует stale для Binance Spot.
4. Запустить обратно → stale_monitor логирует stale_recovered.
5. kill -TERM <pid> — скрипт завершается корректно (graceful_shutdown в логах).
6. Остановить Redis — коллекторы логируют redis_error и продолжают работу. Запустить Redis — redis_recovered.
7. latency_monitor каждые 10 сек логирует latency_report.
8. Все логи парсятся jq без ошибок.
9. Код каждого коллектора: < 250 строк. common.py: < 100 строк.

10. Сознательно исключено

- Docker / docker-compose — запуск напрямую через python.
- Prometheus metrics — вместо них structlog JSON.
- Hot-reload символов (SIGHUP) — вместо этого перезапуск скрипта.

- Micro-batching — поштучная запись pipeline достаточна.
- Abstract base classes — каждый скрипт самодостаточен.
- HTTP health endpoint — логи достаточны для мониторинга.
- Grafana dashboards — JSON-логи покрывают потребности.

Любой из этих пунктов может быть добавлен позже без переписывания архитектуры.

— Конец документа —